

## Shiv Nadar University Chennai

|                     |                                                            |                                |
|---------------------|------------------------------------------------------------|--------------------------------|
| Name of Student     | <b>KR Srinidhi Rajamane</b>                                | Reg. No.<br><b>24011101059</b> |
| Degree & Branch     | B.Tech Artificial Intelligence & Data Science<br>Section A | Semester V                     |
| Subject Code & Name | CS3807 – Deep Learning Laboratory                          | AY: 2026–<br>27                |

### **Experiment 1** **Implementation of a Single Layer Perceptron for Binary Classification**

#### **1. Objective**

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

#### **2. Tasks to be Performed**

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

#### **3. Background Theory**

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

## Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$  : Input vector
- $\mathbf{w}$  : Weight vector
- $b$  : Bias
- $z$  : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where  $\eta$  denotes the learning rate.

## 4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

### Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

| Activation | Output Range        | Typical Applications                    |
|------------|---------------------|-----------------------------------------|
| Step       | $\{0, 1\}$          | Classical Perceptron                    |
| Sigmoid    | $(0, 1)$            | Binary Classification Output Layer      |
| Tanh       | $(-1, 1)$           | Hidden Layers (Traditional Networks)    |
| ReLU       | $[0, \infty)$       | Hidden Layers in CNNs and MLPs          |
| Leaky ReLU | $(-\infty, \infty)$ | Avoiding Dying ReLU                     |
| Softmax    | $(0, 1)$            | Multi-class Classification Output Layer |

**Note:** This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

## 5. Dataset Description

**Dataset:** Banknote Authentication Dataset

**Source:** UCI Machine Learning Repository

**Download Link:**

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

|                |                       |
|----------------|-----------------------|
| Instances      | 1372                  |
| Features       | 4 Numerical Features  |
| Classes        | 2                     |
| Missing Values | None                  |
| Task           | Binary Classification |

### Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

## 6. Experimental Procedure

### Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

### Expected Output

Dataset summary and statistical information.

### Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

### Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

### Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

### Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

### Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

## 7. Mandatory Plots

| Plot                         | Purpose                | Expected Inference                                                |
|------------------------------|------------------------|-------------------------------------------------------------------|
| Feature Histograms           | Distribution Analysis  | Identify skewness and outliers.                                   |
| Correlation Heatmap          | Feature Relationship   | Observe highly correlated features.                               |
| Scatter Plot                 | Class Distribution     | Determine whether classes are approximately linearly separable.   |
| Training Error vs Epoch      | Learning Behaviour     | Verify convergence of the perceptron.                             |
| Weight Evolution             | Parameter Analysis     | Observe stabilization of learned weights.                         |
| Bias Evolution               | Parameter Analysis     | Understand the role of the bias term.                             |
| Confusion Matrix             | Performance Evaluation | Interpret TP, FP, TN and FN.                                      |
| Learning Rate Comparison     | Hyperparameter Study   | Compare convergence behaviour for different learning rates.       |
| Decision Boundary (Optional) | Visualization          | Illustrate the separating hyperplane using two selected features. |

## 8. Performance Tables

### Training Summary

|                  |                                                                                        |
|------------------|----------------------------------------------------------------------------------------|
| Dataset Size     | 1372 samples                                                                           |
| Train/Test Split | 80% / 20% (1097 train / 275 test)                                                      |
| Learning Rate    | 0.01                                                                                   |
| Epochs           | 50 (ran for the full budget; did not reach zero error)                                 |
| Final Weights    | Variance: $-0.1637$ , Skewness: $-0.1767$ ,<br>Curtosis: $-0.1895$ , Entropy: $0.0040$ |
| Final Bias       | $0.2400$                                                                               |
| Accuracy         | $0.9709$ (97.09%)                                                                      |
| Precision        | $0.9385$ (93.85%)                                                                      |
| Recall           | $1.0000$ (100.00%)                                                                     |
| F1-score         | $0.9683$ (96.83%)                                                                      |

### Epoch-wise Learning

(Weight 1 = Variance, Weight 2 = Skewness; selected epochs shown for brevity)

| Epoch | Errors | Weight 1  | Weight 2  | Bias     |
|-------|--------|-----------|-----------|----------|
| 1     | 175    | $-0.0744$ | $-0.0491$ | $0.0700$ |
| 5     | 32     | $-0.0964$ | $-0.0952$ | $0.1200$ |
| 10    | 36     | $-0.1125$ | $-0.1215$ | $0.1500$ |
| 20    | 20     | $-0.1231$ | $-0.1465$ | $0.1800$ |
| 30    | 22     | $-0.1389$ | $-0.1609$ | $0.2000$ |
| 40    | 18     | $-0.1499$ | $-0.1795$ | $0.2200$ |
| 47    | 16     | $-0.1642$ | $-0.1847$ | $0.2300$ |
| 50    | 27     | $-0.1637$ | $-0.1767$ | $0.2400$ |

## 9. Additional Tasks

1. Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.
2. Compare the implementation with Scikit-learn's Perceptron.
3. Repeat the experiment using learning rates 0.001, 0.01 and 0.1.
4. Explain why a Single Layer Perceptron cannot solve the XOR problem.
5. Study the effect of feature normalization on model convergence.

## 10. Source Code

The complete implementation was carried out in Python using NumPy, Pandas, Matplotlib, Seaborn and Scikit-learn. The full, executable source code (organized task-wise, exactly as it was run) is available on GitHub:

[https://github.com/Srinidhi-Krishna/Deep-Learning-Lab/blob/main/Lab\\_1/DL\\_LAB\\_1.ipynb](https://github.com/Srinidhi-Krishna/Deep-Learning-Lab/blob/main/Lab_1/DL_LAB_1.ipynb)

## 11. Experimental Results

### 11.1 Dataset Exploration (Task 1)

The dataset loaded fine – 1372 rows and 5 columns (4 features + class label). `df.isnull().sum()` showed zero missing values anywhere, so no imputation needed. Class split is 762 authentic vs 610 forged, fairly balanced, no resampling required. From `df.describe()`, Skewness has the widest spread (std  $\approx 5.87$ , range roughly  $-13.77$  to  $12.95$ ) while Entropy is the most tightly packed (std  $\approx 2.10$ ). That gap in scale is basically the reason normalization was needed before training.

### 11.2 Data Preprocessing (Task 3)

After `MinMaxScaler`, all four features got squeezed into  $[0, 1]$  (checked the printed min/max: 0.000 and 1.000). Stratified 80/20 split gave 1097 train / 275 test samples, class ratio preserved on both sides.

### 11.3 Model Training (Task 5)

Trained for a fixed 50 epochs,  $\eta = 0.01$ . Epoch 1 started with 175 errors, dropped fast to around 30–40 by epoch 10, then just kept bouncing between roughly 16 and 30 for the rest of training instead of settling down. Lowest point was epoch 47 with 16 errors, but it climbed again right after. Along with the epoch table in Section 8, this basically confirms the four features together aren't perfectly linearly separable – the perceptron never locks onto a zero-error line in 50 epochs, it just keeps circling a decent solution.

### 11.4 Model Evaluation (Task 6)

Final-epoch weights on the test set gave:

| Metric    | Value            |
|-----------|------------------|
| Accuracy  | 0.9709 (97.09%)  |
| Precision | 0.9385 (93.85%)  |
| Recall    | 1.0000 (100.00%) |
| F1-score  | 0.9683 (96.83%)  |

Confusion matrix on the 275 test samples:

$$\begin{bmatrix} 145 & 8 \\ 0 & 122 \end{bmatrix}$$

145 authentic notes correctly caught, 8 authentic notes wrongly flagged as forged, and zero forged notes slipped through as authentic. That's why recall on the forged class is a perfect 1.0 – for something like currency checking, that's the error type you actually want to avoid.

### 11.5 Effect of Learning Rate (Task 7)

Tried three rates, each trained fresh for 50 epochs:

| Learning Rate ( $\eta$ ) | Epochs Run | Final Misclassified |
|--------------------------|------------|---------------------|
| 0.001                    | 50         | 16                  |
| 0.01                     | 50         | 27                  |
| 0.1                      | 50         | 27                  |

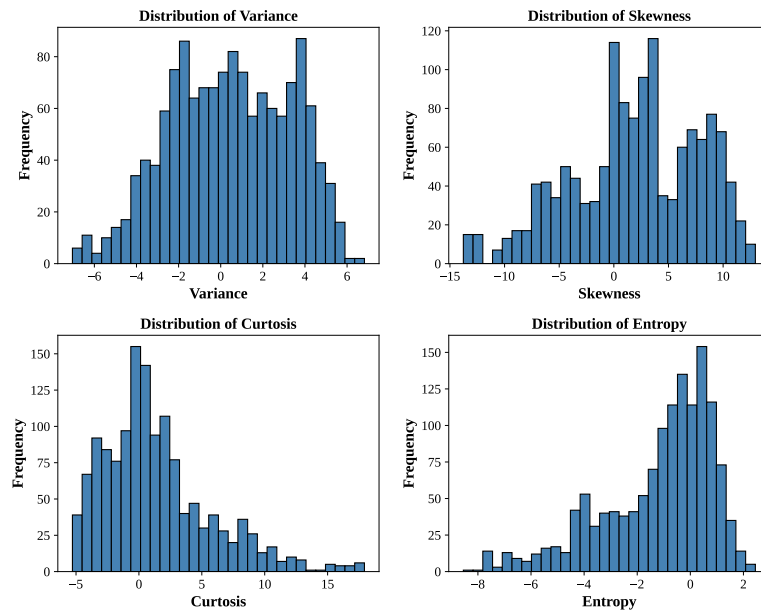
Bit surprising – I expected the smallest rate to be the slowest one, but  $\eta = 0.001$  actually ends up with the fewest errors. Small steps mean the weight vector moves cautiously and doesn't overshoot, while  $\eta = 0.01$  and 0.1 take bigger corrective jumps and keep bouncing the boundary around instead of settling, which is a known thing with the perceptron rule on data that isn't perfectly separable.

### 11.6 Comparison with Scikit-learn (Task 9, Optional)

Ran scikit-learn's Perceptron with the same settings (`max_iter=50`, `eta0=0.01`) on the same split and got identical numbers to the from-scratch version – 0.9709 accuracy, 0.9385 precision, 1.0000 recall, 0.9683 F1 for both. Good sign that the weight update, step activation and forward pass were coded correctly.

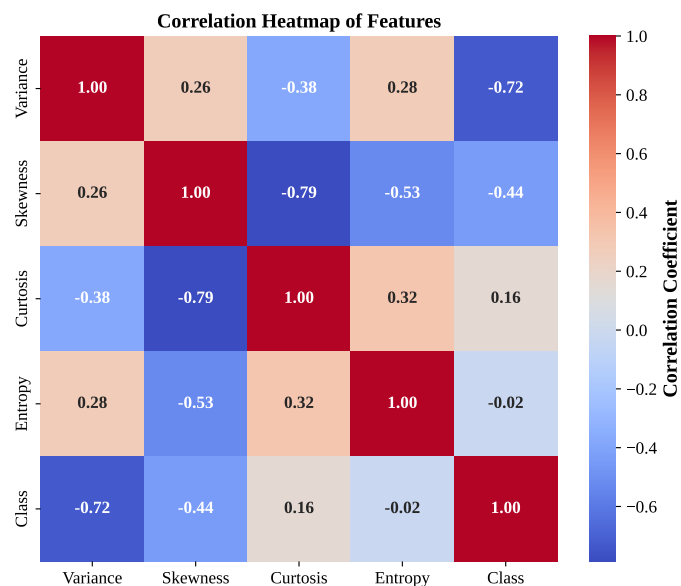


## 12. Generated Plots with Interpretation



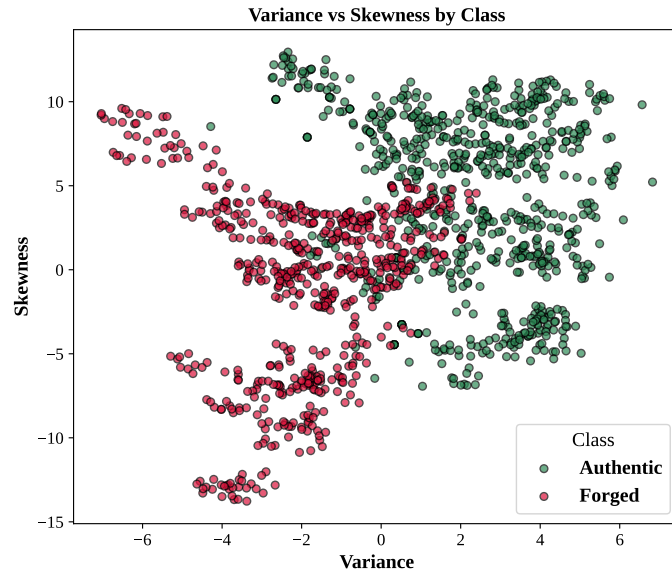
**Figure 1:** Frequency distribution histograms of Variance, Skewness, Kurtosis and Entropy.

**Interpretation:** None of the four features look properly bell-shaped – Skewness and Kurtosis have a longer tail on one side and Entropy leans more toward negative values. Raw features aren't normally distributed, which doesn't matter much for a perceptron, but it's the reason I went with min-max scaling instead of standardization.



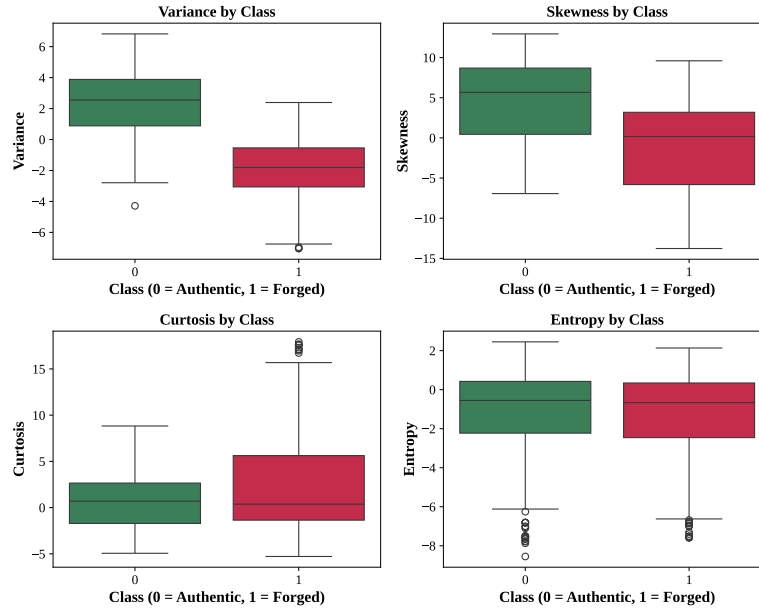
**Figure 2:** Pearson correlation heatmap among the four features and the class label.

**Interpretation:** Variance has the strongest correlation with Class out of the four, which lines up with the final weights – Variance, Skewness and Kurtosis all get decent-sized negative weights while Entropy stays near zero. Variance and Kurtosis are also correlated with each other, so there's some redundancy, but not enough to make either feature useless.



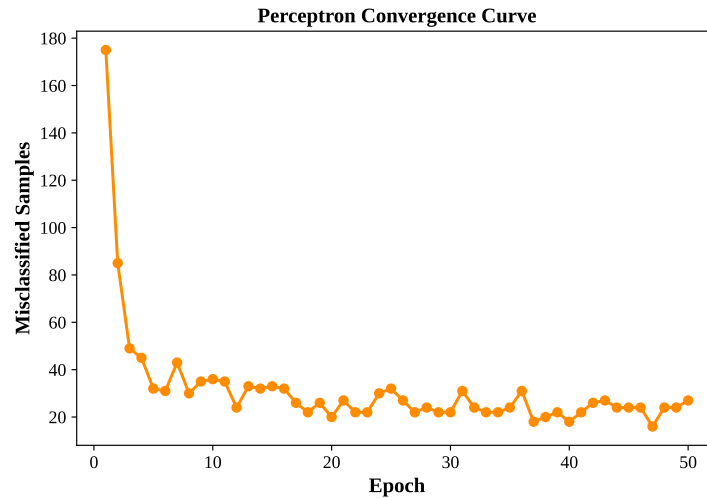
**Figure 3:** Scatter plot of Variance versus Skewness, colour-coded by class.

**Interpretation:** This plot is what convinced me a linear classifier could actually work here – authentic and forged points mostly sit in different regions, with authentic notes leaning toward higher Variance and Skewness. There's a band where the two classes overlap though, and that overlap is probably what's stopping the perceptron from hitting zero training error.



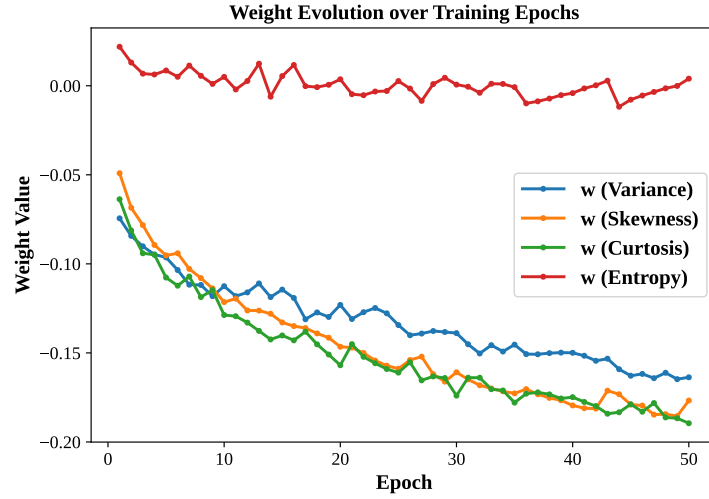
**Figure 4:** Boxplots of each feature grouped by class.

**Interpretation:** Same story as the scatter plot – median Variance and Skewness for authentic notes sit above forged notes, with only partial overlap in the IQRs. Kurtosis shows some separation too but smaller, and Entropy’s boxes almost overlap completely, matching it having the smallest final weight.



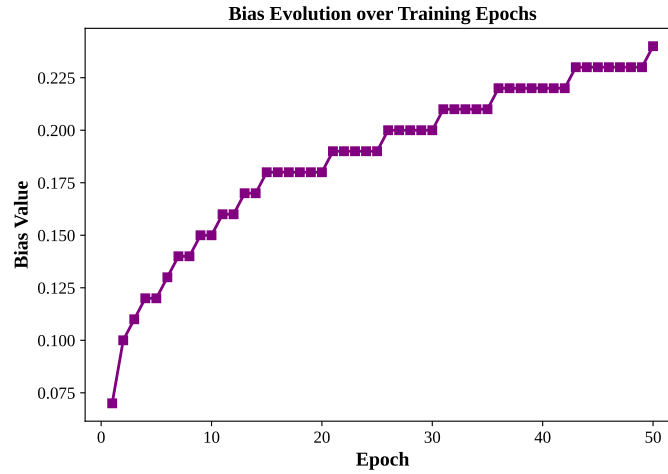
**Figure 5:** Misclassified training samples per epoch (learning rate 0.01).

**Interpretation:** Errors drop fast for the first 8–10 epochs then flatten into a noisy plateau between about 16 and 30 for the rest of training. It never hits zero, which by the Perceptron Convergence Theorem means the four features together aren’t perfectly linearly separable – if they were, the algorithm would eventually find a zero-error boundary.



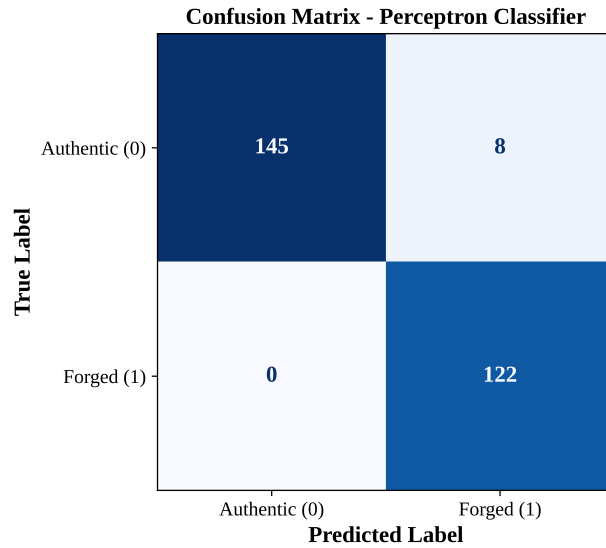
**Figure 6:** Evolution of the four feature weights across training epochs.

**Interpretation:** All four weights move fast early on then level off into small oscillations instead of a flat line, matching the fact that error never hits zero. Variance, Skewness and Kurtosis settle near similar negative values while Entropy stays close to zero the whole time – Entropy is basically the weakest feature here.



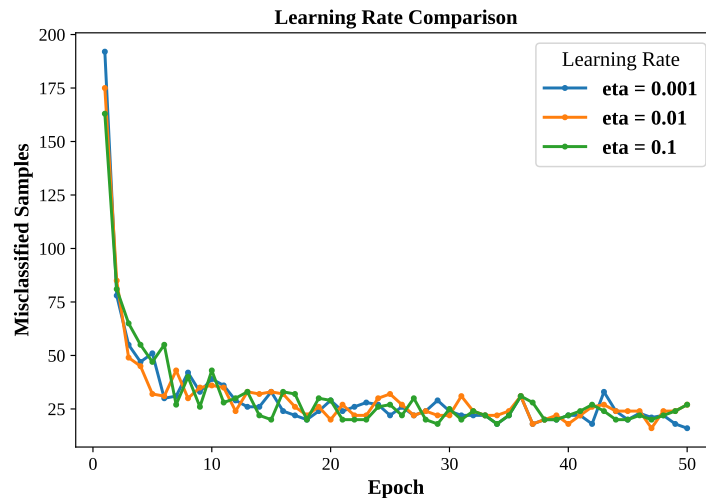
**Figure 7:** Evolution of the bias term across training epochs.

**Interpretation:** The bias climbs in a rough staircase pattern from 0 up to about 0.24. Since bias only updates on a misclassification, each step is basically a batch of errors nudging the boundary further, and it's still climbing at epoch 50, which means the model is still making corrections right up to the end.



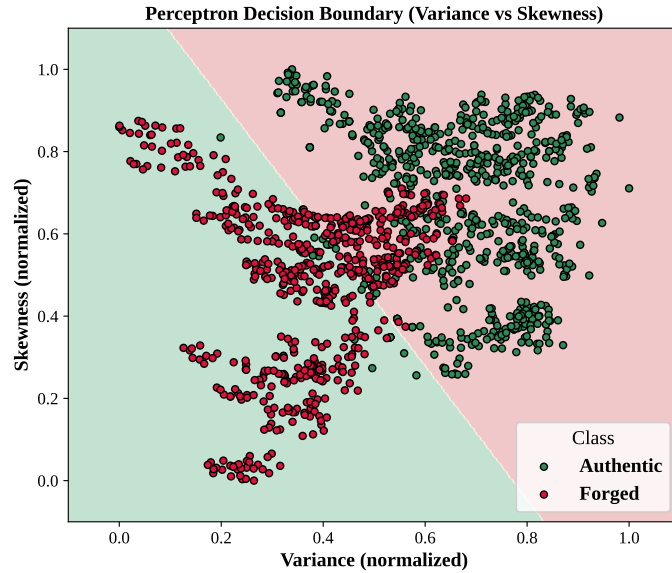
**Figure 8:** Confusion matrix on the held-out test set (275 samples).

**Interpretation:** Only 8 out of 275 test notes were misclassified, and all 8 were authentic notes flagged as forged – no forged note got through as authentic. For currency verification that’s a good error pattern to have, since a false alarm on a genuine note is a lot less costly than letting a fake one pass.



**Figure 9:** Convergence behaviour of the perceptron under three different learning rates.

**Interpretation:** All three curves drop fast at first but settle at different noise floors –  $\eta = 0.001$  ends up lowest at 16 errors while  $\eta = 0.01$  and  $0.1$  both plateau around 27. This is basically what Section 11.5 is about: smaller steps here land closer to a good boundary, larger steps keep the weight vector jumping around more.



**Figure 10:** Decision boundary learned by a 2-feature perceptron (Variance vs Skewness).

**Interpretation:** With just two features the boundary is easy to see – most authentic points on one side, most forged on the other, with a few points from each class right on or just past the line. Confirms the same limitation from the Additional Tasks: the boundary can only ever be a straight line, so any overlap that isn’t linearly separable will always leave a few points misclassified.

## 13. Discussion

The from-scratch perceptron does well here – 97.09% test accuracy and, more importantly for a fraud-detection kind of task, 100% recall on the forged class. It never let a fake note through as genuine on the test split.

That said, the training curves (Figs. 5, 7, 8) make it clear the model never fully converges in 50 epochs – the error count keeps oscillating in the high teens/twenties instead of hitting zero. Combined with the class overlap visible in Figs. 3 and 4, this suggests the four features are only approximately linearly separable, not perfectly, so there’s a small pocket of points near the boundary the perceptron can never get right no matter how long it trains (the Convergence Theorem only guarantees a zero-error solution when the data actually is separable).

The learning-rate result in 11.5 is worth pointing out too – the smallest rate ( $\eta = 0.001$ ) ended up with fewer errors than the larger ones, which goes against the usual “bigger step = faster learning” intuition. On non-separable data, a large learning rate can keep overshooting a good region instead of settling into it, while a small rate moves more conservatively and can land closer to an optimum even though individual updates are tiny.

Normalization mattered too – Skewness has a much bigger raw scale (std  $\approx 5.87$ ) than Entropy (std  $\approx 2.10$ ), so without scaling, Skewness would dominate the weighted sum just because of its magnitude, not because it’s more informative.

On the activation function: Step is what makes this a “classical” perceptron, but it’s also why the algorithm can’t be extended to deeper networks – its derivative is zero everywhere except at  $z = 0$  where it’s

undefined, so there's no gradient to push backward through multiple layers. That's the whole reason Sigmoid/Tanh/ReLU exist – to give every layer a smooth, non-zero gradient so stacked layers can actually be trained.

Same reasoning explains why a single-layer perceptron can't solve XOR: XOR isn't linearly separable (no single line separates  $(0, 1)/(1, 0)$  from  $(0, 0)/(1, 1)$ ), and one perceptron can only draw one linear boundary. XOR needs at least one hidden layer with a non-linear activation to remap the inputs into a space where the classes do become separable.

Finally, matching scikit-learn's `Perceptron` exactly under the same settings is a good sign – the weight update rule, stopping condition and forward pass all seem to be implemented correctly, not just getting lucky numbers.

## 14. Conclusion

Implemented a Single Layer Perceptron from scratch and tested it on the UCI Banknote Authentication dataset. Got 97.09% accuracy, 93.85% precision, 100% recall and 96.83% F1 on the test set, and the output matched scikit-learn's built-in `Perceptron` exactly under the same hyperparameters – a good check that the training loop, weight updates and step activation were coded right.

The training curves showed the four features are only approximately linearly separable, so the model settles into a noisy low-error region instead of reaching zero, and the learning-rate experiment showed a smaller step size can actually converge better than a larger one on this kind of data.

Beyond the numbers, this experiment made it clear why the Step activation – fine for one linear boundary – can't be used in deeper networks, and why something like XOR needs more than one layer to solve. Overall I think the goals of the experiment were met: understanding the perceptron rule, seeing its training dynamics visually, and checking it against standard metrics and a reference library.

## 15. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.